

1. Extended Methodology

1.1 Data Sources

Stage 1 (game level win model). Two team level files from Moore (n.d.) were used.

- TeamStatistics.csv has one row per team per game. It holds score, box score counts (assists, blocks, steals, field goals, three-pointers, free throws, rebounds, turnovers, fouls), quarter and overtime points, bench points, biggest lead, biggest scoring run, fast-break, paint and second-chance points, timeouts remaining, season win-loss record, coach, and playoff seed.
- TeamStatisticsExtended.csv has advanced metrics: offensive, defensive, and net ratings; assist, rebound, and turnover percentages; effective and true shooting; pace and possessions; points off turnovers; opponent points (paint, fast break, second chance, off turnovers); shot mix and assisted/unassisted percentages; free-throw rate; and opponent four factor measures.

Stage 2 (player level projection). Six sources were used.

1. nba_api (swar/nba_api, n.d.) provided LeagueDashPlayerStats, LeagueDashPlayerBioStats, LeagueDashPlayerShotLocations, LeagueDashPtDefend, LeagueDashPlayerPtShot, GravityLeaders, and ScheduleLeagueV2, for 2021-22 through 2026-27. GravityLeaders and the bio stats were retrieved but not used in the final feature set.
2. Miscellaneous team context statistics were also used (points off turnovers, second-chance, fast-break, and paint points, plus opponent versions) [TODO: confirm source, e.g., nba.com/stats].
3. Rosters for the projected season came from nba.com.
4. NBA 2K player ratings came from nba2kapi (n.d.).
5. ESPN depth charts were transcribed manually into a dictionary of positional depth lists.
6. Preseason Vegas win totals and title odds, and final standings, were entered manually as dictionaries (win totals: Rogers, n.d.)

1.2 Data Collection

The Stage 1 CSVs were loaded directly. For Stage 2, a loop called each nba_api endpoint once per season (regular season data only; the schedule endpoint with league_id="00") and saved the

result to a local CSV named {yy}_{yy}{Endpoint}.csv. A loader parsed those filenames with a regular expression and rebuilt a nested dictionary, data[season][endpoint]. This cached the API pulls so later steps ran on local files. Cleaning steps operate on in memory copies, so the stored files remain the raw originals.

1.3 Data Cleaning

Stage 1. Each game should appear as exactly two rows, one per team, and the notebook checks this in three ways.

- Game IDs present in the standard file but absent from the extended file were identified (Figures S1–S2). From 1997 onward these comprised 78 team game rows (39 games): 74 rows from 2005–06 and 4 rows from the 2023 and 2024 NBA Cup championship games. These games were removed.
- Rows with a missing team and opponent name were dropped. These were two spurious rows, with game IDs 22500651 and 22500652, carrying a placeholder team ID of 0.
- A pairwise symmetry check confirmed that each consecutive pair of rows contained opposing teams with consistent IDs. After cleaning, every game had exactly two rows and every pair matched.

Missing game_Type labels were filled in stages. First, games played September–March were labeled Regular Season and games in May–June were labeled Playoffs. Second, manual rules covered known edge cases: the April 12–15, 2022 play-in window, the April 16–June 16, 2022 playoffs, regular season games from April 1–10, 2022, and the July 22–30, 2020 bubble seeding games. Third, exhibition and international opponents (for example, "Team Austin", "Team Melo", "Stripes", "Stars", "Jerusalem B.C.") were labeled All-Star Game/Other. A final check confirmed that no games lacked a type. A season year was derived from the date (games in October or later take that calendar year, earlier games take the prior year). The resulting composition by season and game type is shown in Figure S3.

Stage 2. Rank columns, fantasy ranking columns, and redundant tracking columns (age, games played, field goal totals that duplicate the stats table) were dropped. Columns were renamed for consistency (for example, the defensive tracking fields were prefixed D_). The shot location table's unnamed header columns were relabeled as player and team identifiers. Missing shot location and miscellaneous values were set to zero, since a missing value there means no recorded attempts.

1.4 Dataset Merging

Stage 1. An initial approach that put winners and losers side by side in one wide row was abandoned, and the code is retained but commented out. The adopted approach iterates over the extended file in consecutive pairs. For each pair, it matches both rows to the standard file using a

composite key of (gameId, teamId, teamName). It then runs the same game and opposing team sanity checks and emits two team perspective rows per game. Each row contains the team's standard and extended statistics plus, for every statistic, a differential (diff_*) equal to the team's value minus the opponent's. The result is one dataframe of team game rows.

Stage 2. Within each season, the tables were joined on player ID. LeagueDashPlayerStats was joined to shot locations, then defensive tracking (joined on D_PLAYER_ID), then shot frequency. The miscellaneous table was joined on a normalized player name, because it has no ID column. Name normalization harmonized suffixes ("Jr." to "Jr") and accented characters (for example, Valančiūnas, Čančar, Quiñones).

For 13 players, the tables are taken from the 2024-25 season in every other season: Lillard, Haliburton, Irving, Butler, Morant, Young, Davis, VanVleet, Sabonis, Jackson Jr., LaVine, Edey, and Kessler

1.5 Duplicate and Missing Data

Stage 1. The extended-era team table contained 79,688 rows with at least one missing value among the columns checked. Missingness is concentrated in a few fields.

- About 7,200 rows lack bench points, biggest lead, fast-break points, paint points, second-chance points, quarter points, timeouts, and season record.
- About 14,400 rows lack the rebound-percentage fields.
- Overtime point fields are largely missing because most games have no overtime.

Values were not imputed. XGBoost handles missing values natively during the exploratory feature set comparisons. The final model then used only features with no missing values (the seven differentials listed in 1.7).

1.6 Feature Engineering

Stage 1. Every team game row carries the raw team value and a team minus opponent differential for each statistic. The final model used the differentials because they encode the matchup directly: a positive value always means the team out-produced its opponent in that category.

Stage 2. Counting statistics were converted to per game averages by dividing the season total by games played, and ratio and percentage fields were left unchanged. Three scoring by zone features were built from shot location made field goal counts.

- Paint points = $2 \times (\text{restricted area makes} + \text{non restricted area paint makes})$.
- Mid-range points = $2 \times \text{mid-range makes}$.

- Three-point points = $3 \times (\text{left corner} + \text{right corner} + \text{above-the-break} + \text{backcourt makes})$.

NBA 2K heights were converted to inches. Positions were mapped to numbers (PG=1, SG=2, SF=3, PF=4, C=5) for the rookie model and then collapsed to guard/forward/center labels (PG and SG to G, SF and PF to F, C to C) for rotation building.

Team features for game prediction are the sums of the seven model features across the nine man rotation, different between the two opponents (see 1.15–1.16).

1.7 Statistical Categories and Features

Stage 1 category models:

- Six category specific models were trained first, once on raw team statistics and once on differentials.

Category	Raw feature version	Differential version
Shooting	FGA, 3PM, 3PA, FTA, FT%, PFDrawn, %Pts 2PT, %Pts mid-range, %Pts 3PT, %Pts FT	FGA, 3PM, 3PA, 3P%, FTM, FTA, FT%, FTA rate, PFDrawn, %2PT FGA, %3PT FGA, %Pts 2PT, %Pts mid-range, %Pts 3PT, %Pts FT
Rebounding	OReb, DReb, opponent second-chance pts	OReb, DReb, TReb, team rebounds, OReb%, DReb%, Reb%, second-chance pts, opponent second-chance pts
Scoring	%Pts fast break, %Pts off TO, %Pts in paint	same three, plus fast-break pts, paint pts, bench pts, biggest scoring run
Defense	Blk, Stl, PF, opponent score, opponent fast-break and paint pts, opponent eFG%, opponent FTA rate, opponent TO%, pts off TO, opponent second-chance pts	Blk, Stl, PF, pts off TO, opponent pts off TO, opponent fast-break, paint, and second-chance pts, opponent FTA rate, opponent TO%, opponent OReb%

Pace	Pace	Pace, estimated pace, pace per 40, possessions
Playmaking	Ast, TO, team TO, AST%, A/TO, %assisted 2PM, 3PM, FGM	Ast, TO, team TO%, AST%, A/TO, %assisted and %unassisted 2PM, 3PM, FGM

Final Stage 1 model (seven differentials). Defensive rebounds, assists, points off turnovers, three-pointers made, free throws made, opponent points in the paint, and blocks. These features were chosen using SHAP rankings and PDP shapes from the category models, and all seven are fully populated in the data.

1.8 Stage 1 Model Setup and Hyperparameters

All Stage 1 models are XGBoost classifiers ("XGBoost," 2026) with logistic loss and eval_metric="logloss". The target is the binary Win label. Category model performance is reported in Table S1.

Model	colsampl e	learning rate	max depth	n_estimator s	subsampl e	Other
All features (Attempt 1)	1.0	0.2	7	200	1.0	early stopping (25)
Shooting	1.0	0.05	3	600	0.8	
Rebounding	0.8	0.10	3	600	0.8	
Scoring	0.8	0.30	3	200	0.8	
Defense	0.8	0.03	3	600	0.8	
Pace	0.8	0.10	3	200	1.0	
Playmaking	0.8	0.05	3	200	0.8	
Final (7 features)	0.5	0.005	6	300	0.6	min_child_weight=5, gamma=0.2, early stopping (25)

Category models used `random_state=1` and were reused unchanged for the differential versions. The final model's parameters were selected by hyperparameter search, and the notebook stores only the selected values. Earlier parameter sets tried on the smaller feature set (learning rate 0.10 to 0.15, depth 3 to 4, 150 to 200 trees) were superseded. The final model was trained with early stopping on validation log loss, evaluating on both the training and validation sets.

1.9 Train/Validation/Test Methodology

The Stage 1 modeling table contains regular season rows only. Within each season, rows were shuffled and split into 90% train plus validation and 10% test (`random_state=1`). The train plus validation portion was split again into 80% training and 20% validation, giving about 70/20/10 per season. Splitting within a season keeps every era (including the shortened 1998 and 2011 seasons) represented in all three sets. Because the split was applied to team game rows and each game contributes two rows, the two rows of one game can fall in different splits (Figures S3–S4). Playoff games were never used for training and served as a separate out of sample check.

Stage 2's rookie regressors used a fixed 80/20 split of veterans with an NBA 2K match (`random_state=42`). The same players were used as the test set for every target, so the whole rookie system could be evaluated consistently. Targets with fewer than 20 training samples were skipped.

1.10 Model Evaluation

Stage 1. Models were evaluated on the held out test set with accuracy (0.5 threshold), confusion matrix, ROC AUC, log loss, and Brier score. The final model was also evaluated with a ROC curve, a 10 bin calibration curve, a row normalized confusion matrix, and a train versus validation log loss curve by boosting round (Figures S30–S33). It was additionally scored on all playoff games with the same metrics.

Stage 2. The rookie regressors were scored by MAE, RMSE, and R^2 on held-out veterans. The projection pipeline was evaluated by comparing simulated team win totals with preseason Vegas over/unders and with actual wins (1.17).

1.11 SHAP and Partial Dependence Methodology

Feature influence was measured with SHAP (Lundberg & Lee, 2017) using TreeExplainer on the test set. Importance is the mean absolute SHAP value per feature, reported as a ranked table, a bar plot, and a beeswarm plot. SHAP values for XGBoost classifiers are in log-odds units. Partial dependence plots (scikit-learn, 30-point grid) show predicted win probability (y-axis fixed at 0–100%) against each feature. They were generated for every feature in each category model (raw and differential versions) and for the final model (Figures S5–S29). The workflow was iterative: the category models' SHAP and PDP results were used to select the most impactful and reliably populated features, which were then combined in the final seven feature model.

1.12 Player Projection Methodology

For each player, the projection baseline is per game production from the season's stat files, using the 2024-25 season for the 13 override players in 1.4. No age curve or regression to the mean is applied. Stage 2 is therefore a roster and availability model built on carried forward production rather than a statistical forecast of individual improvement or decline.

Roster changes were applied by matching nba.com roster names to the stat table and overwriting team assignments. Unmatched players were assigned to "FA" (team ID 0). Rostered players missing from the stat table were appended using their 2024-25 row if they had one, or a row of zeros (age 0) if they did not. The zero rows mark rookies and are filled in 1.13.

1.13 Rookie Projections

Rookies, identified by a player ID of 0 after roster matching (66 in the reported run), have no NBA statistics. Names in the stat table and NBA 2K table were normalized (accent removal, lowercasing, punctuation and suffix removal) and left joined. Veterans with a 2K match were used to train one XGBRegressor per target (300 trees, depth 4, learning rate 0.03, subsample 0.8, column sample 0.8, squared error loss, random_state=42). The trained models then predicted each rookie's per game statistics from his 2K attributes, and the predictions were written back into the per game table. 2K position codes were converted to guard/forward/center labels. On held out veterans, the main per game statistics were recovered with R^2 of about 0.70 to 0.84 (for example points 0.84, assists 0.83, field goals made 0.83, minutes 0.69).

1.14 Roster Construction

Rosters are a dictionary keyed by season and team abbreviation, built from the projected season per game table. Manual adjustments were then applied team by team through a dictionary of per player overrides for minutes, games played, or both. Minutes overrides rescale the player's per game counting statistics by the ratio of target to original minutes (for example, a player moved from 34 to 28 minutes has his counting stats multiplied by 28/34). A games played override changes the player's availability count without touching his statistics. These overrides encode expected role changes and missed games, and no player is removed. ESPN's projected depth charts were transcribed and matched to rosters by last name (suffixes removed) to check roster coverage; unmatched names were logged.

1.15 Nine Man Rotation Methodology

For each game, each team's rotation is built from its currently available players (remaining games greater than zero) in six steps.

1. Sort available players by projected minutes and take the top nine.
2. Enforce positional minimums of at least two centers, one guard, and one forward. A player counts toward every letter in his position label, so a "G-F" counts as both. If a

minimum is unmet, the highest-minute unused real player at that position is added, and the lowest-minute rotation player who does not play that position is removed.

3. If no real player is available at that position, a placeholder player is created. Placeholders copy a low-usage veteran's stat line (Dario Šarić for center, Thanasis Antetokounmpo for forward, Kyle Lowry for guard), scaled to the minutes needed to bring that position to 48 minutes (minimum one).
4. If the rotation still has fewer than nine players, real unused players fill the gap, followed by placeholders.
5. Players are sorted by minutes, and the top five are labeled starters.
6. Minutes are rescaled proportionally to sum to 240, and assists, defensive rebounds, three-pointers, free throws, points off turnovers, and zone points are scaled by the same factor. Block totals are not rescaled.

1.16 Season Simulation Methodology

The schedule came from ScheduleLeagueV2 and was processed in chronological order. Preseason and other non-regular-season rows were excluded (74 of 1,274 schedule rows), leaving 1,200 games processed in the reported run. For each game the simulation does the following.

1. Back-to-back flag: a team is flagged if its previous game was on the prior calendar day in Eastern time.
2. Rotations: build and normalize each team's rotation (1.15).
3. Team features: sum the seven model features across each rotation and compute the differential from each team's perspective.
4. Base probabilities: score each perspective with the final XGBoost model.
5. Home-court adjustment: apply a logit-scale shift of +0.20 for the home team and -0.20 for the away team (about 55/45 from an even matchup).
6. Back-to-back adjustment: if exactly one team is on a back-to-back, subtract 0.03 from its probability and add 0.03 to its opponent's.
7. Winner: the team with the higher adjusted probability wins (if both probabilities land on the same side of 0.5, the higher one wins, and exact ties go to the home team). Outcomes are deterministic, with no random sampling.
8. Bookkeeping: update the standings, decrement each rotation player's remaining games, and record the game date.

The code counts how many predictions each adjustment flips (134 games were flipped by the home-court adjustment). The same pipeline can run completed seasons using final games only, but the reported run is the projected season.

1.17 Vegas Comparison Methodology

For each team, simulated wins were shifted two games toward 41 (teams above 41 lose two, teams below gain two). Model wins were then merged with the preseason Vegas over/under and with actual wins by team abbreviation. Differences were computed as Model – Vegas and Actual – Vegas. Absolute errors against actual wins were computed for both the model and the Vegas line, and summarized by mean, median, standard deviation, minimum, and maximum, separately for the Western and Eastern Conferences.

2. Supplemental Results

2.1 Data Quality and Dataset Composition

Missing extended statistics. Some games in the standard team file had no matching record in the extended file. Figure S1 shows these by year across the full dataset. Most gaps fall between the 1950s and 1995, which is before the modeling window, so the check was repeated for later years (Figure S2).

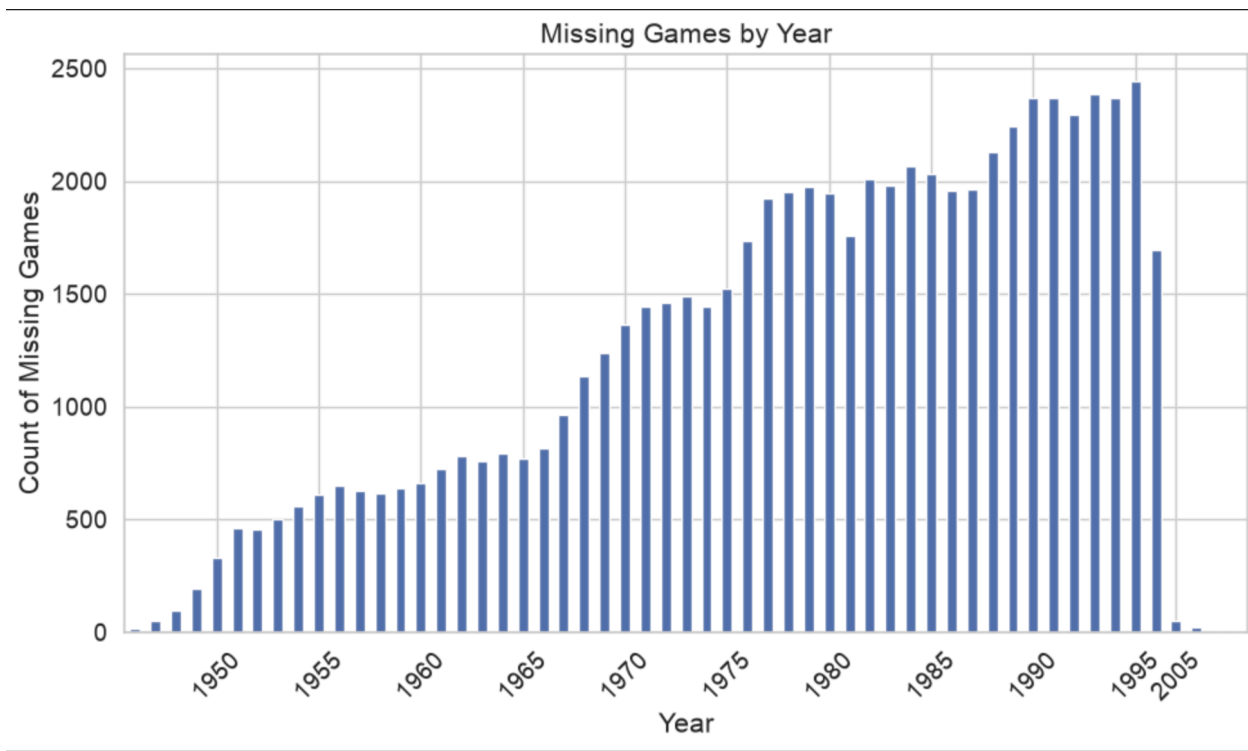


Figure S1: Missing games by year (full dataset)



Figure S2: Missing games by year, 1997 onward

The counts in Figure S2 are team-game rows, and each game has two. The gaps are 2005 (50 rows, 25 games), 2006 (24 rows, 12 games), 2023 (2 rows, 1 game), and 2024 (2 rows, 1 game). In total, 78 rows (39 games) were removed. The 2005–06 games have October dates before the regular season began, and their game IDs fit preseason games. The 2023 and 2024 games are the in-season tournament championship games. Neither group is a regular-season game, so the removal does not affect the modeling set.

Game-type composition. Figure S3 breaks down the unique games in each season from 1996 to 2025 by type: regular season, playoffs, play-in, and All-Star/other. The 1998 and 2011 seasons are visibly shorter because of lockouts, and the play-in category appears only from 2020 onward.

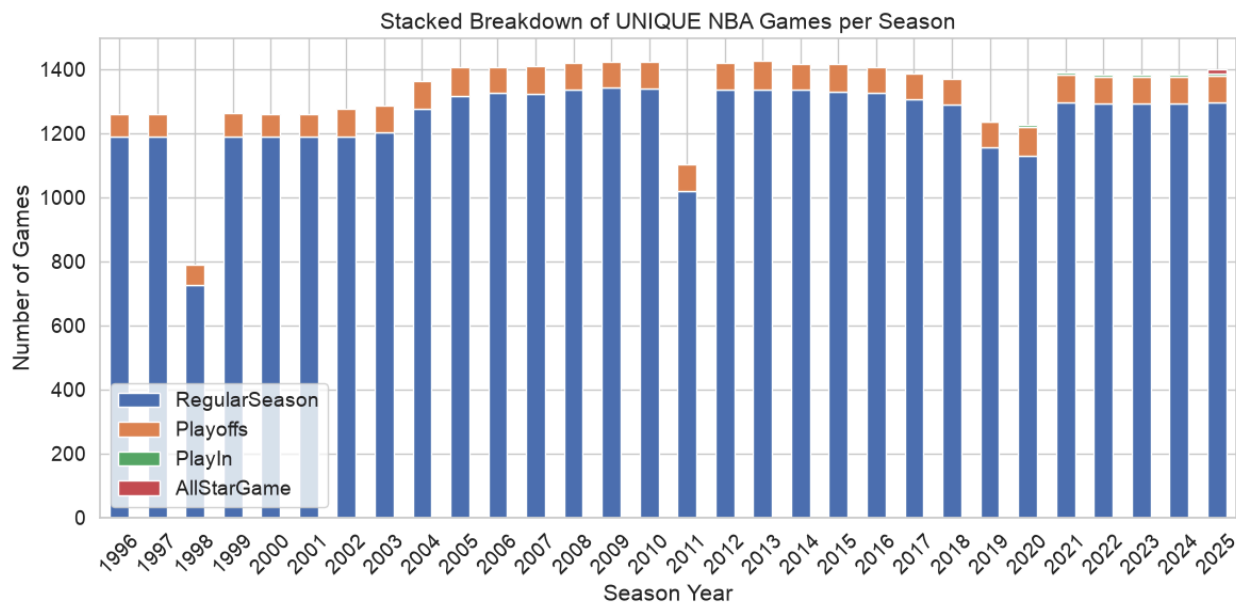


Figure S3: Unique games per season by game type, 1996–2025

Train/validation/test composition. Figure S4 shows, for each season, how many unique games have rows in each split. Only regular-season rows were split.

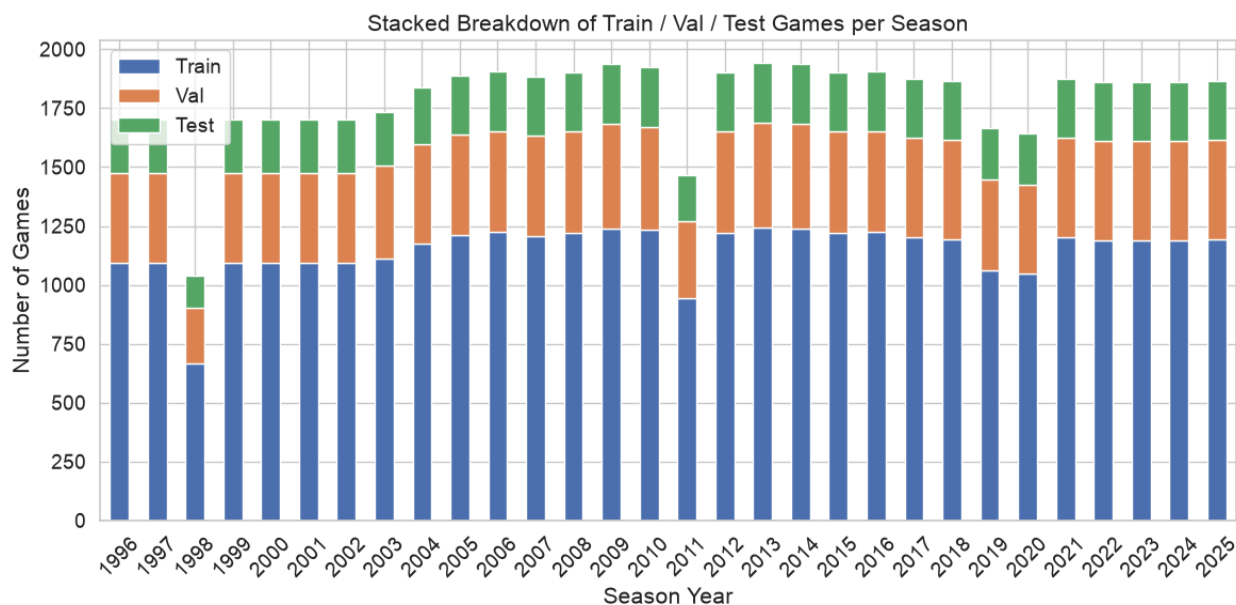


Figure S4: Unique games per season appearing in the training, validation, and test sets

The split was done on team-game rows, not on games. Because each game has two rows, a game can appear in more than one split, so the three bars for a season add up to more than the season's game count. For example, 1996 shows 1,092, 385, and 223 games against 1,189 regular-season games.

2.2 Category-Specific Model Performance

Table S1 gives test-set performance for the six category models trained on raw (non-differential) team statistics. Defense was the most predictive category (AUC 0.831), followed by shooting (0.792), playmaking (0.710), and rebounding (0.694). Scoring was weak (0.587). The pace model scored below chance (accuracy 48.3%, AUC 0.469). This is expected because both teams in a game play the same possessions, so raw pace barely separates winners from losers.

Table S1. Category-specific model results (raw features).

Category	Accuracy	AUC	Log Loss	Brier Score	Most Important Feature
Defense	75.1%	0.8313	0.5041	0.1677	OppEffFG%
Shooting	71.7%	0.7917	0.5539	0.1871	3PM
Playmaking	65.8%	0.7101	0.6242	0.2171	Ast
Rebounding	64.6%	0.6935	0.6338	0.2216	DReb
Scoring	55.6%	0.5872	0.6803	0.2436	%Points_InPaint
Pace	48.3%	0.4693	0.6944	0.2506	Pace

2.3 SHAP and Partial Dependence: Raw-Feature Category Models

Figures S5–S10 show SHAP summary plots (mean absolute SHAP value and feature impact) for each raw-feature category model. Figures S11–S16 show the partial dependence of predicted win probability on each feature in the same models.

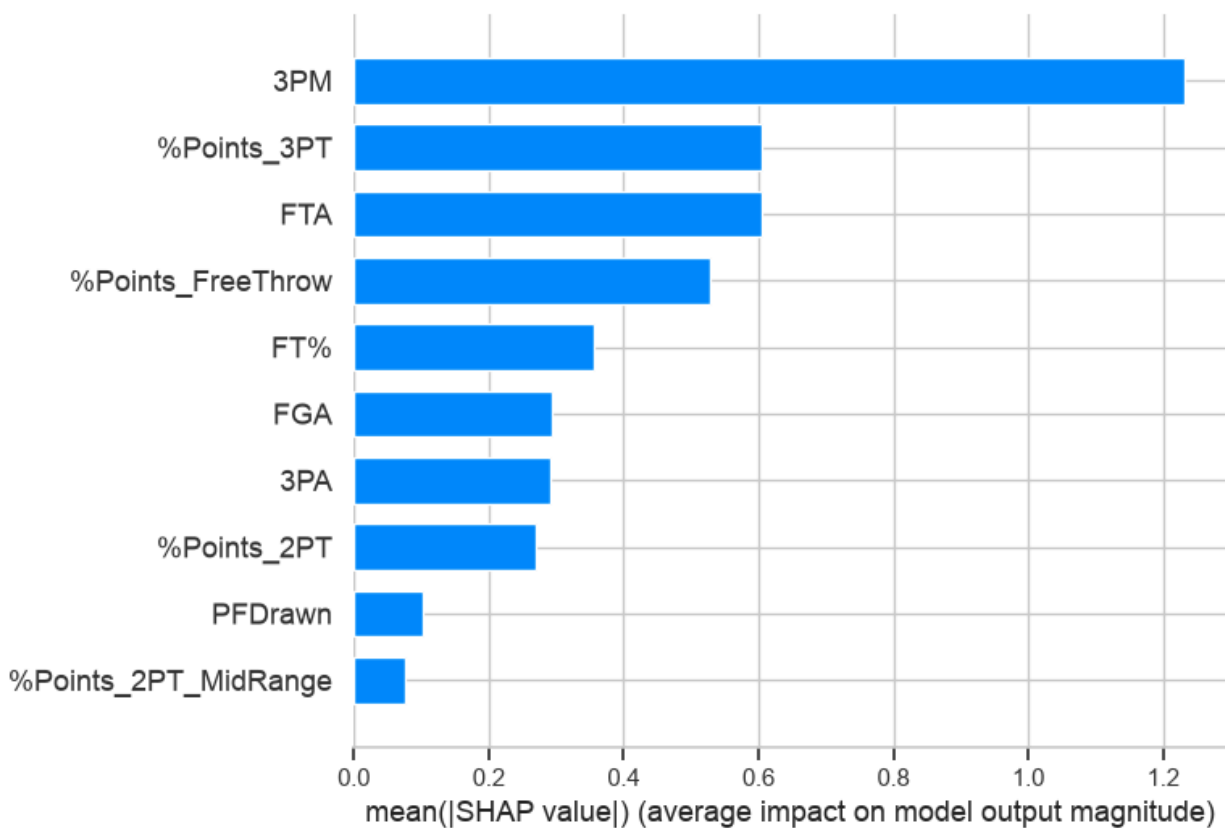


Figure S5: SHAP, shooting model

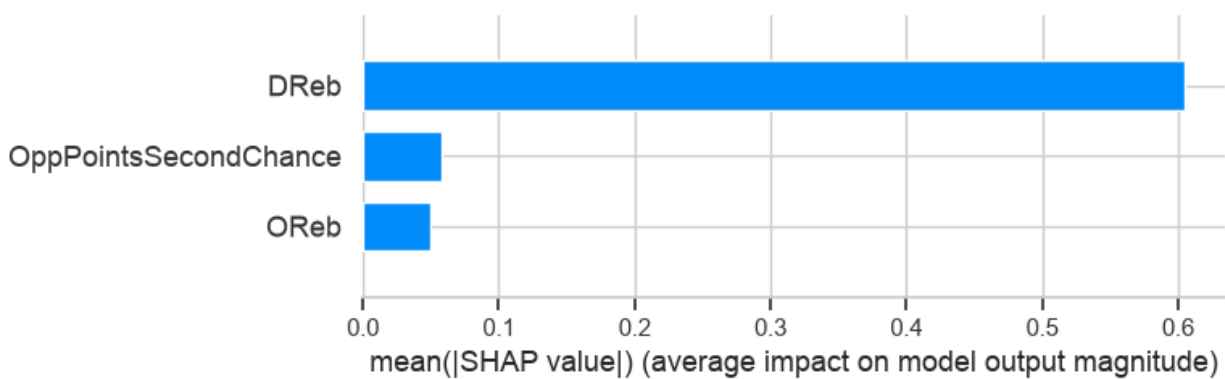


Figure S6 SHAP, rebounding model

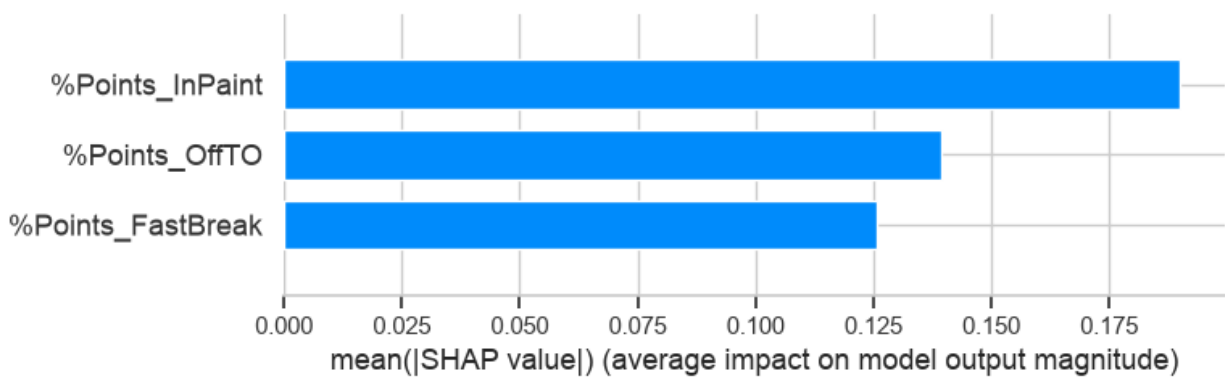


Figure S7: SHAP, scoring model

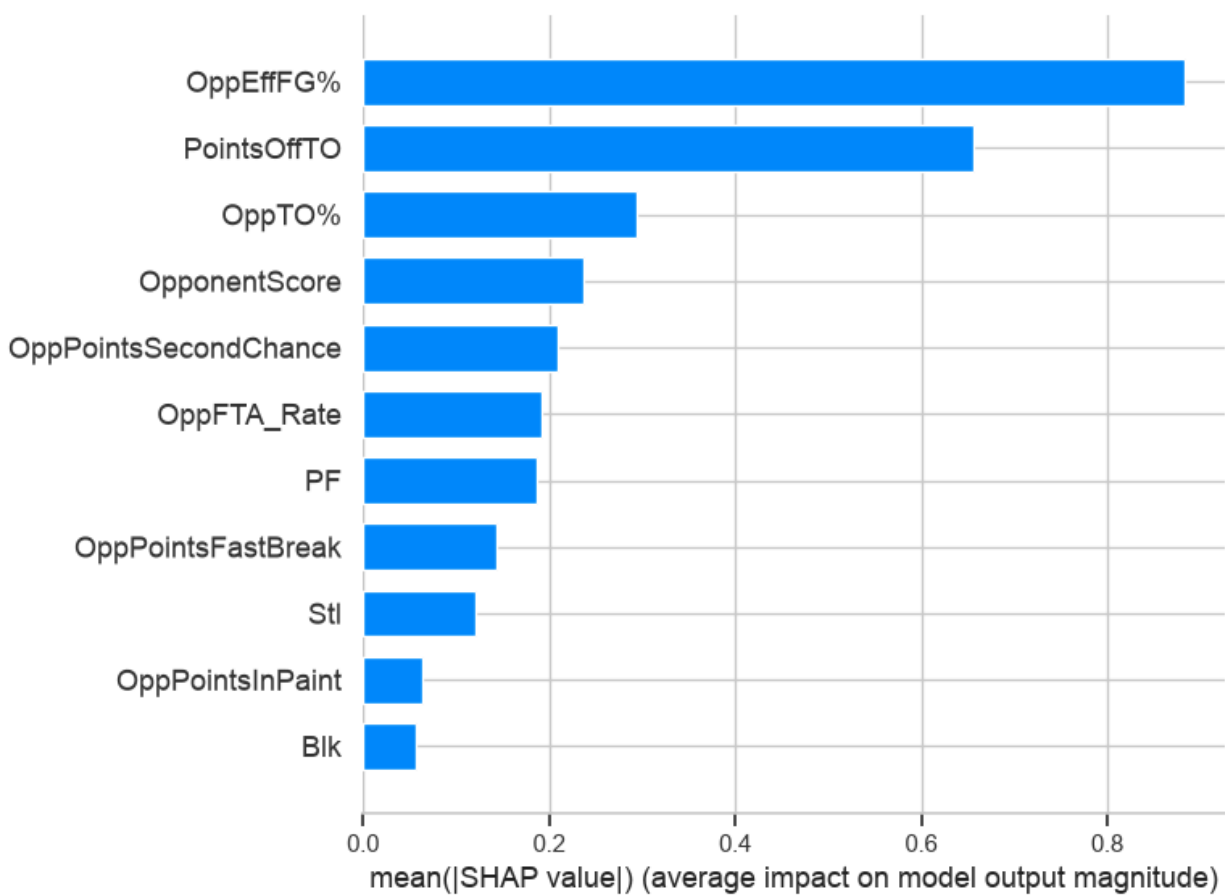


Figure S8: SHAP, defense model

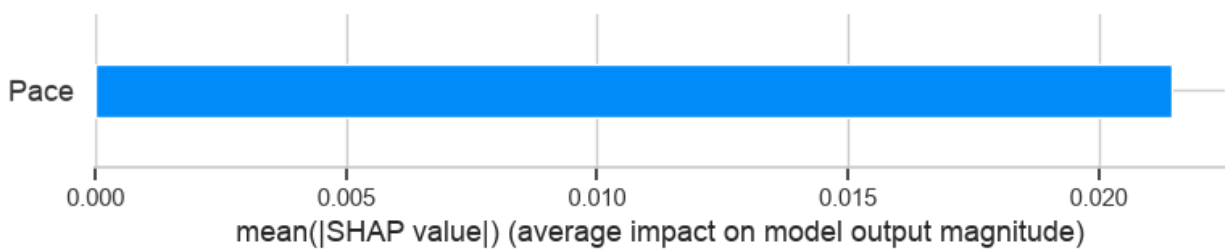


Figure S9. SHAP, pace model.

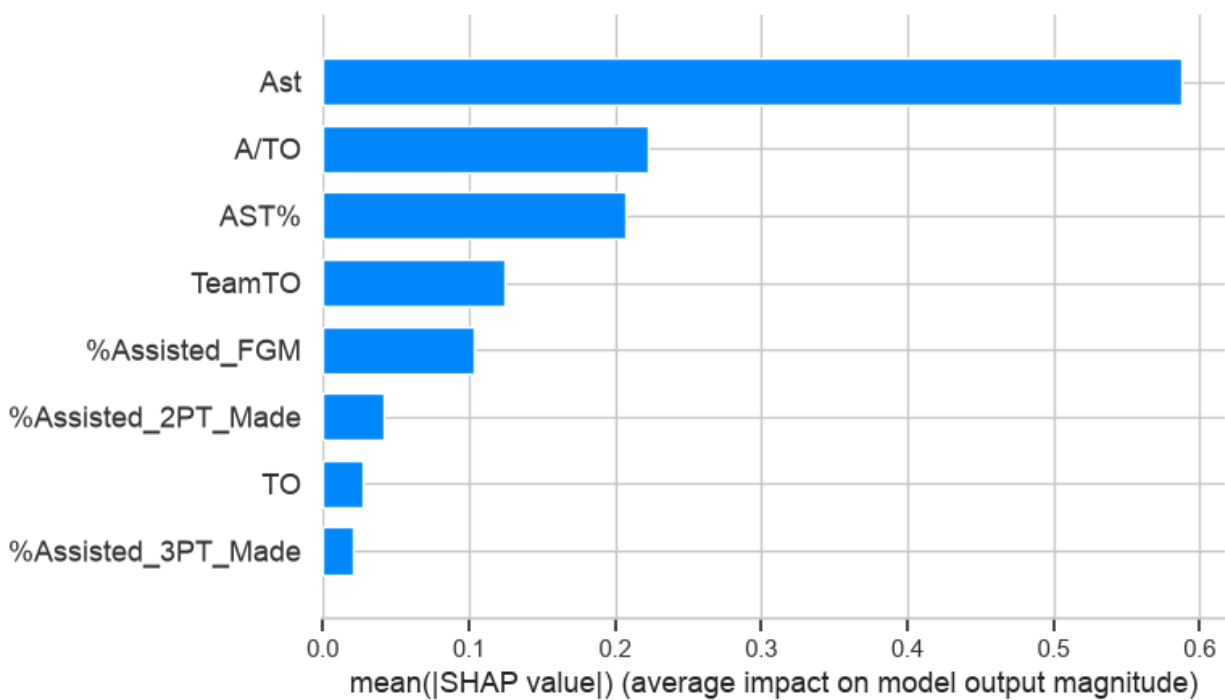


Figure S10. SHAP, playmaking model

Partial Dependence Plots — Shooting Model

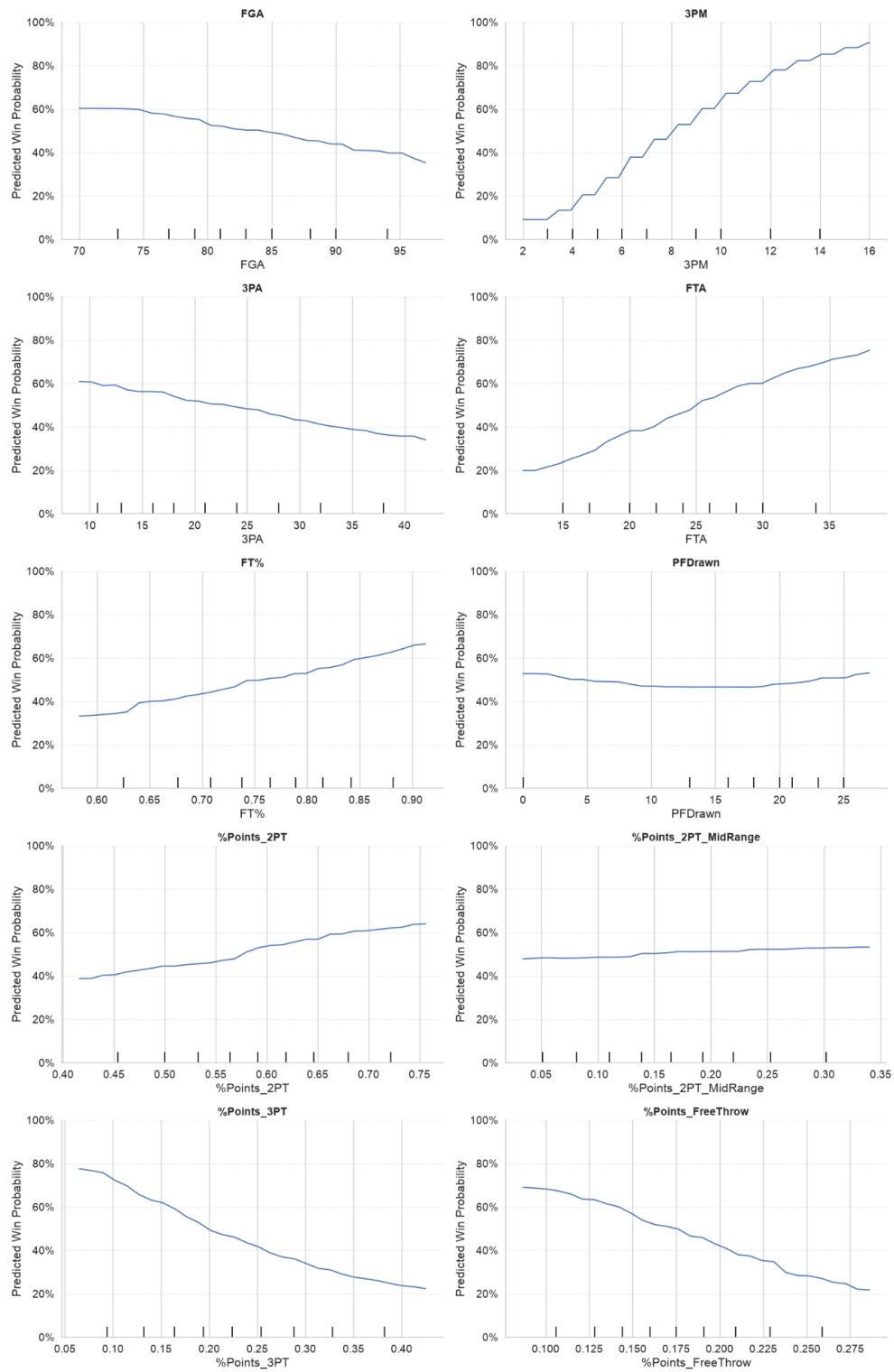


Figure S11. PDP, shooting model

Partial Dependence Plots — Rebounding Model

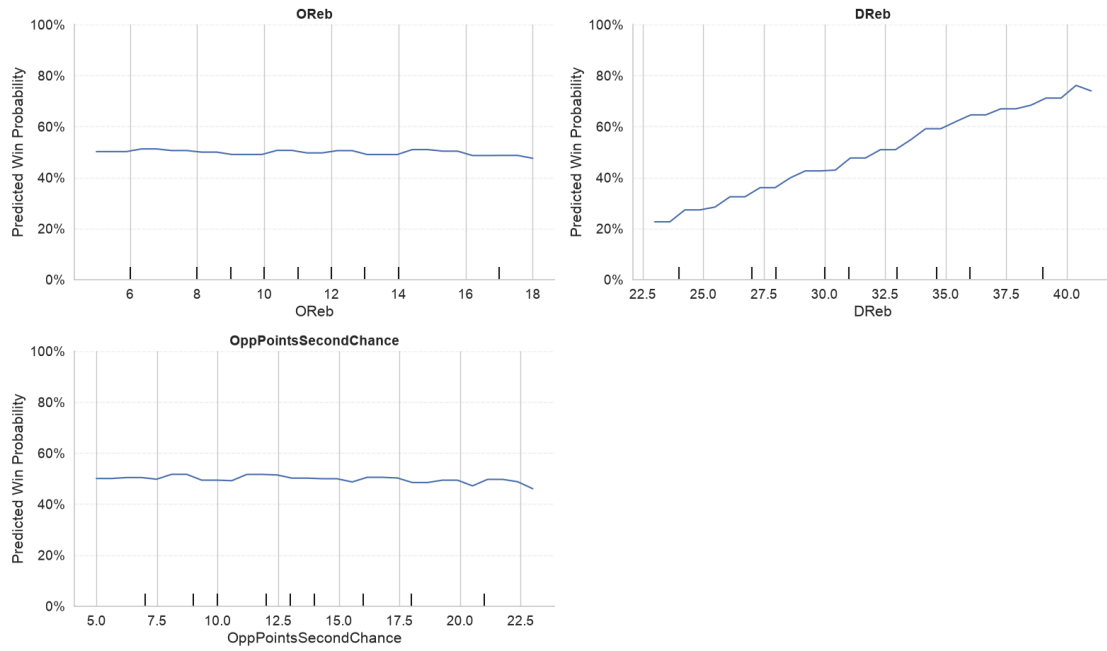


Figure S12. PDP, rebounding model.

Partial Dependence Plots — Scoring Model

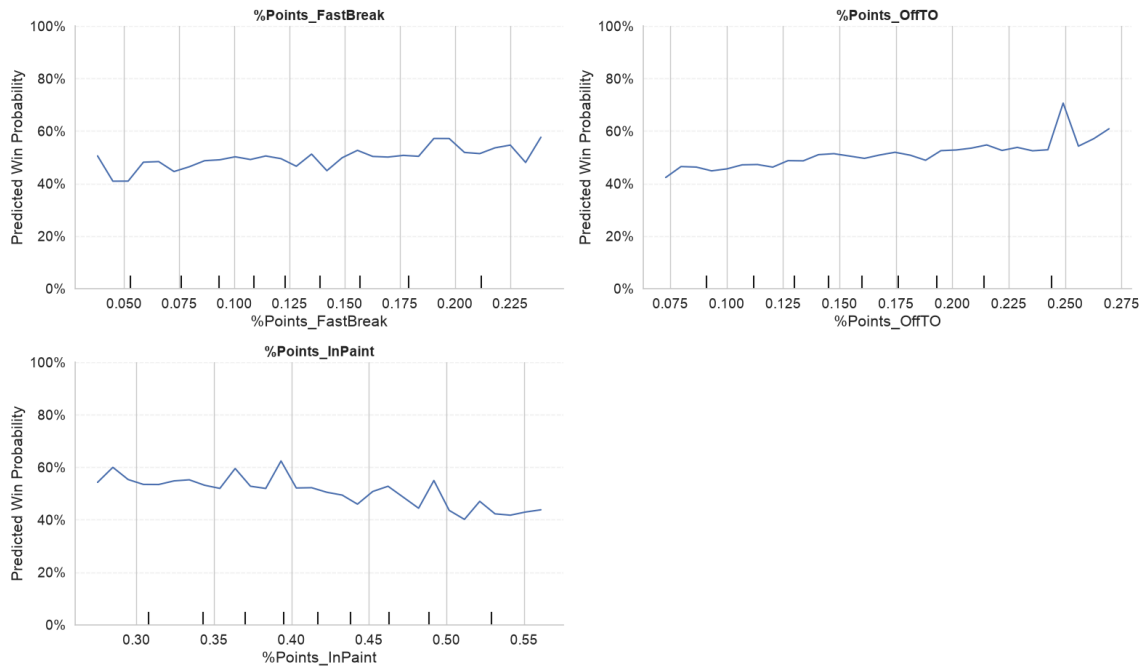


Figure S13. PDP, scoring model

Partial Dependence Plots — Defense Model

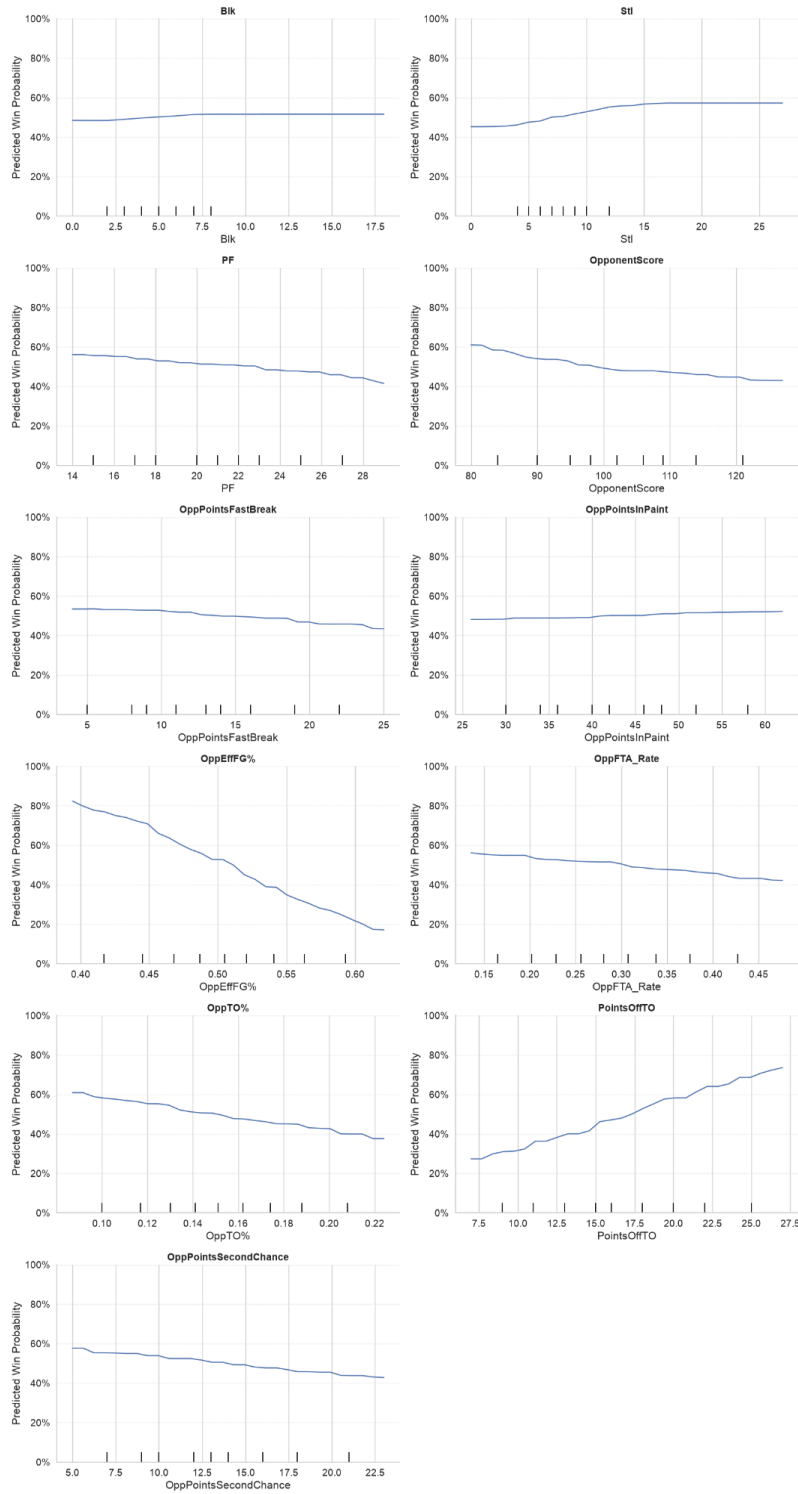


Figure S14. PDP, defense model

Partial Dependence Plots — Pace Model

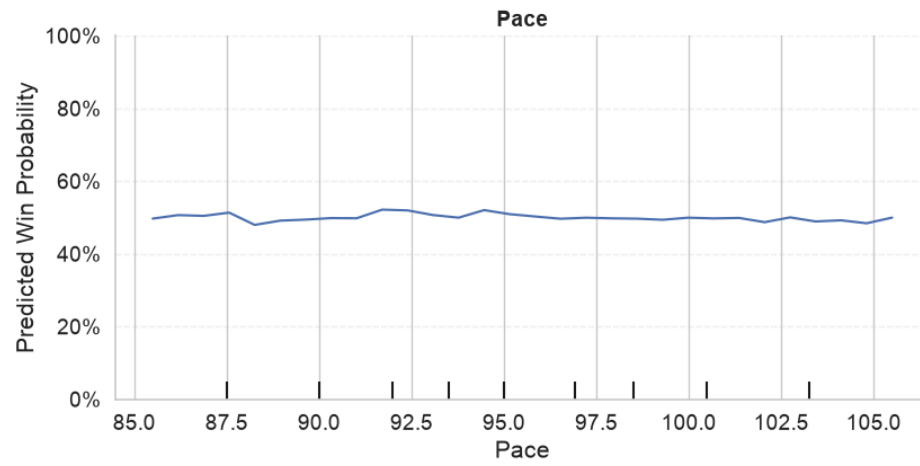


Figure S15: PDP, pace model

Partial Dependence Plots — Playmaking Model

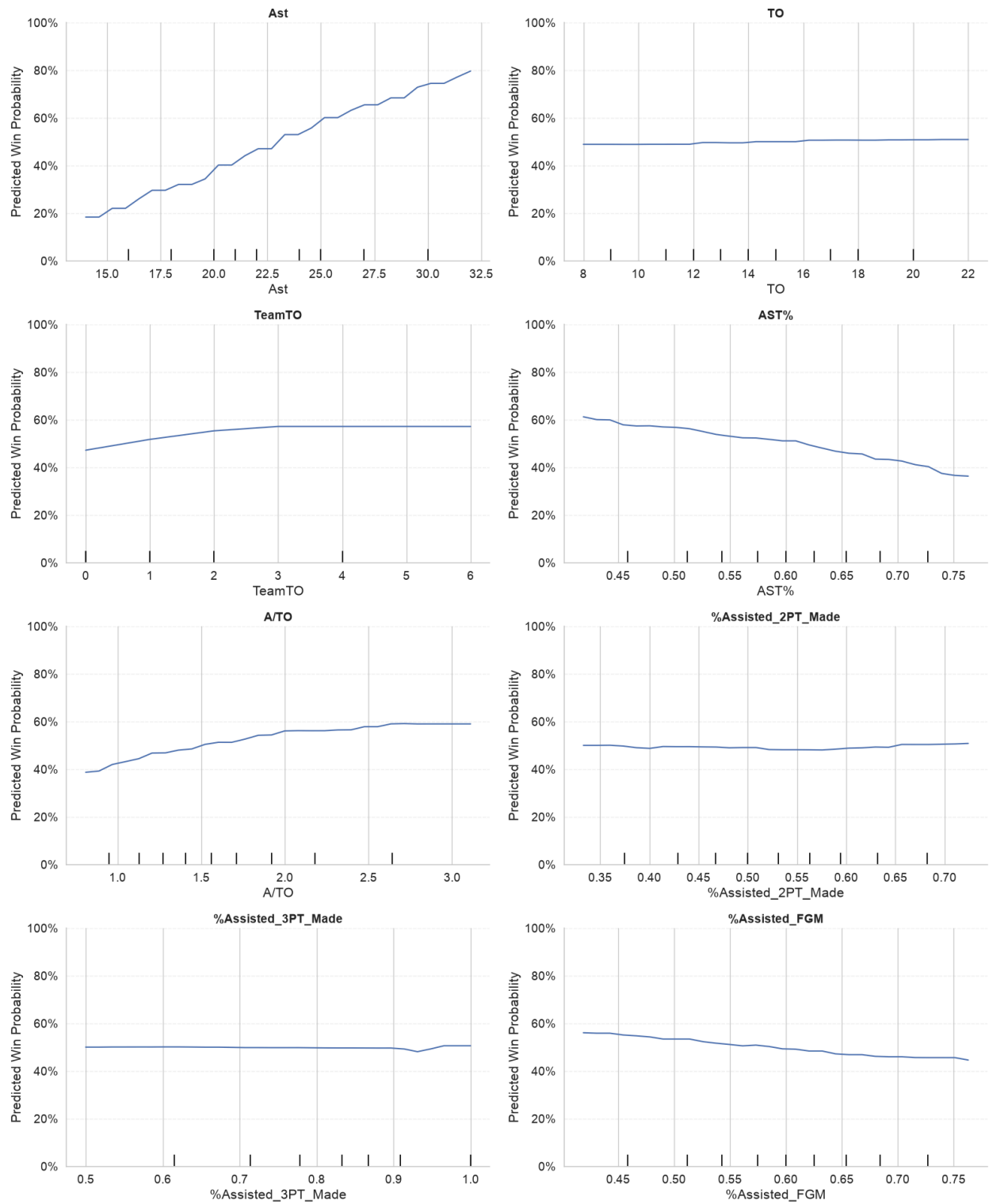


Figure S16: PDP, playmaking model

2.4 SHAP and Partial Dependence: Differential-Feature Category Models

The same six categories were refitted on team minus opponent differentials, reusing the hyperparameters tuned for the raw versions. Figures S17–S22 show their SHAP summaries and Figures S23–S27 show the PDPs

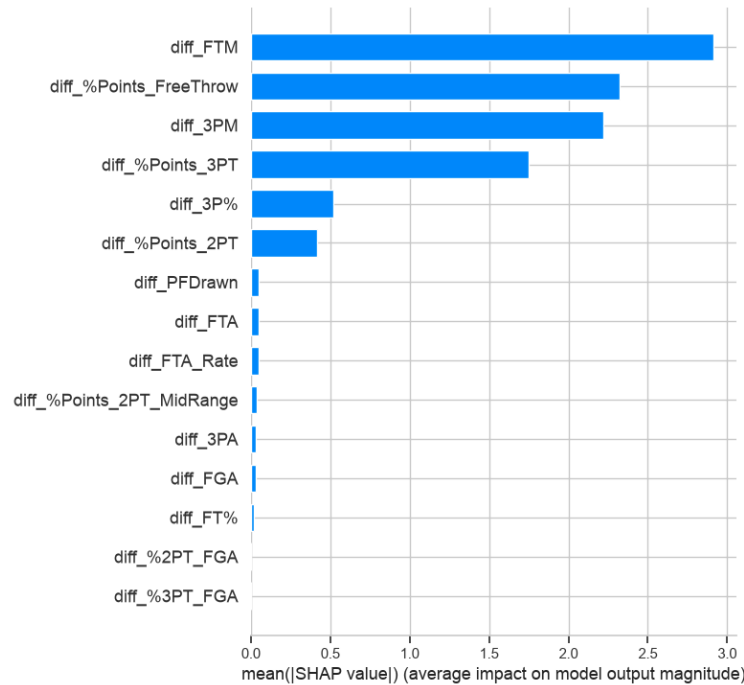


Figure S17: SHAP, shooting (differential).

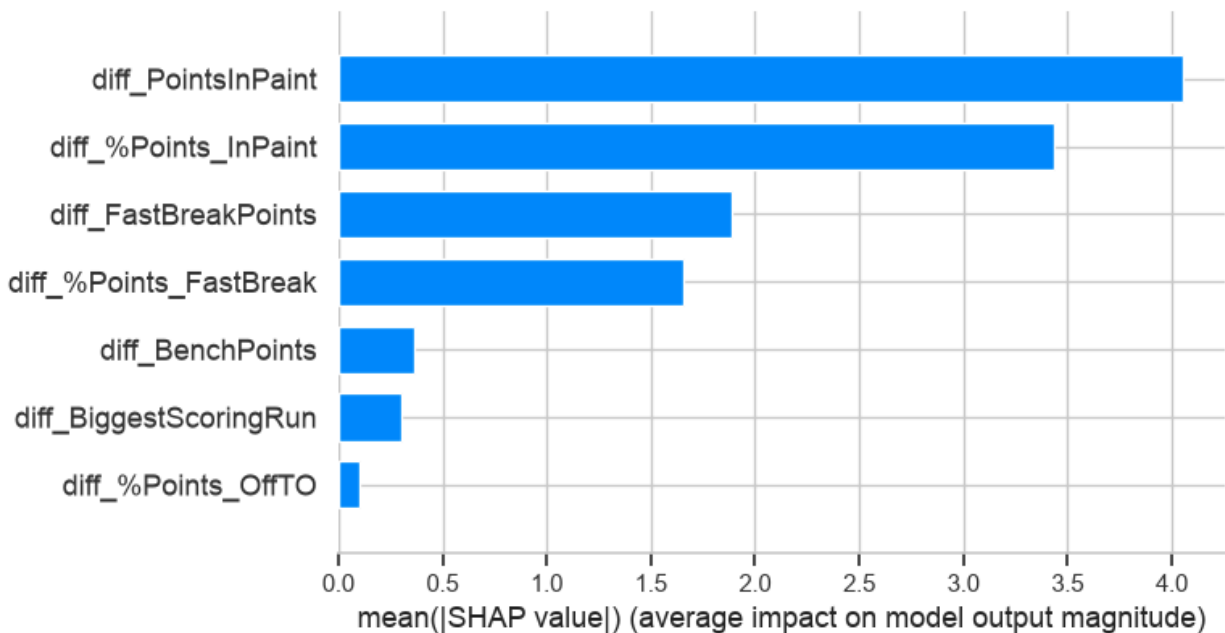


Figure S19: SHAP, scoring (differential)

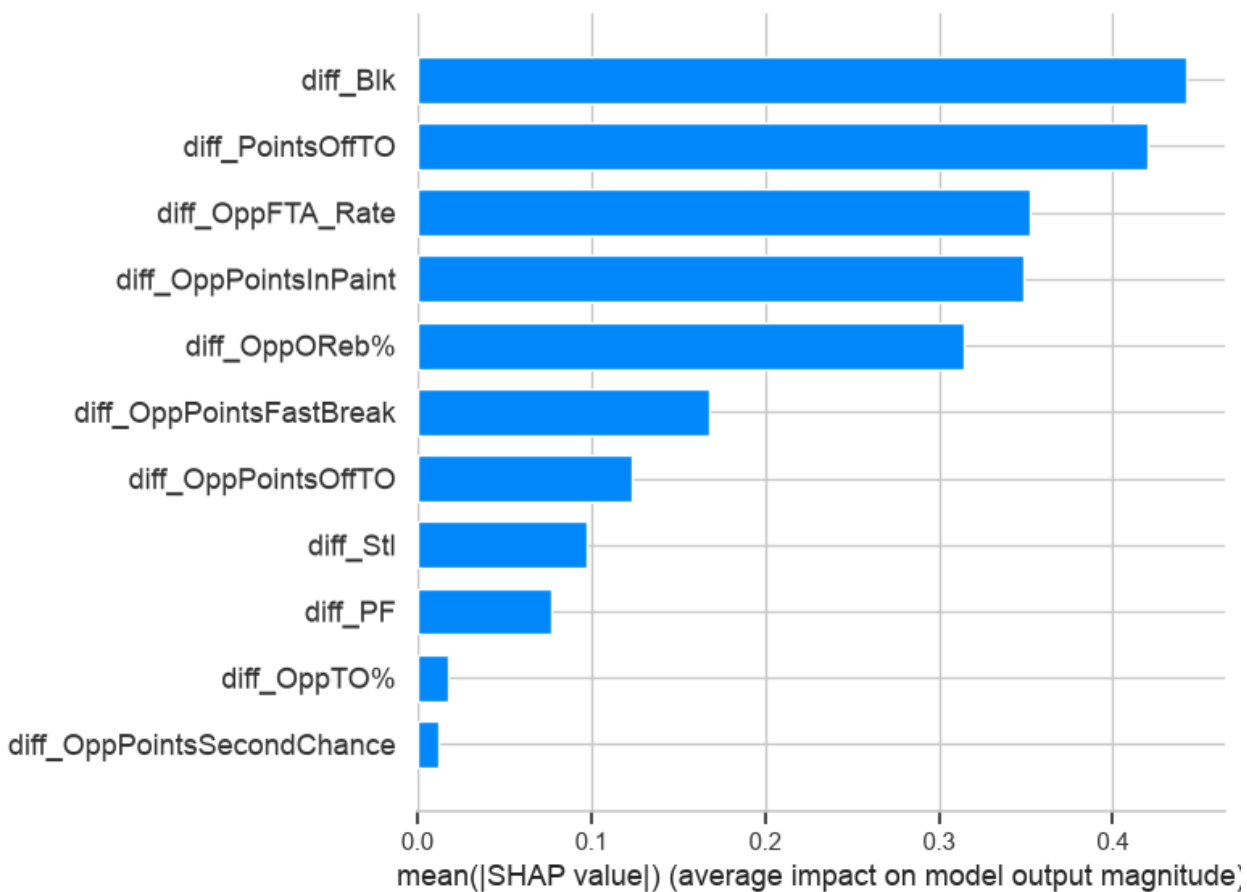


Figure S20: SHAP, defense (differential)

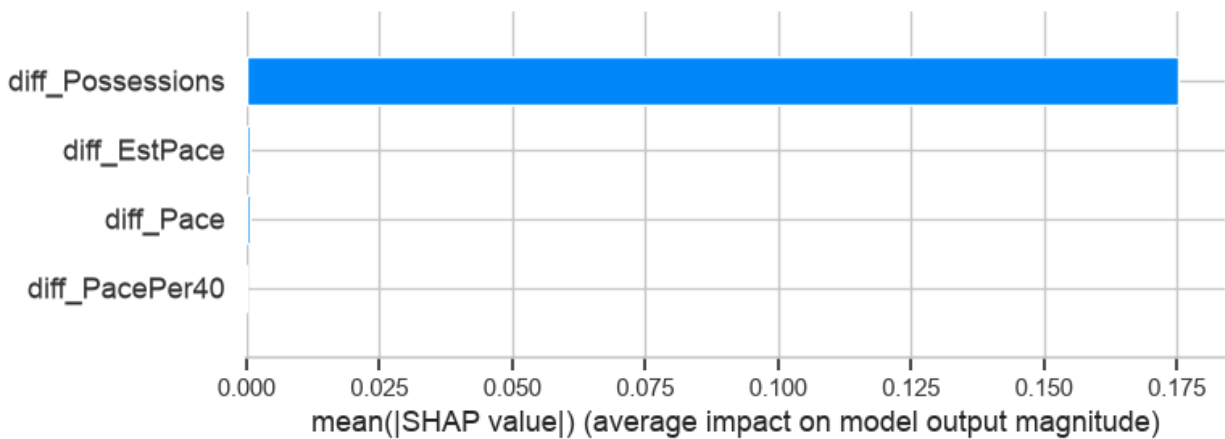


Figure S21: SHAP, pace (differential)

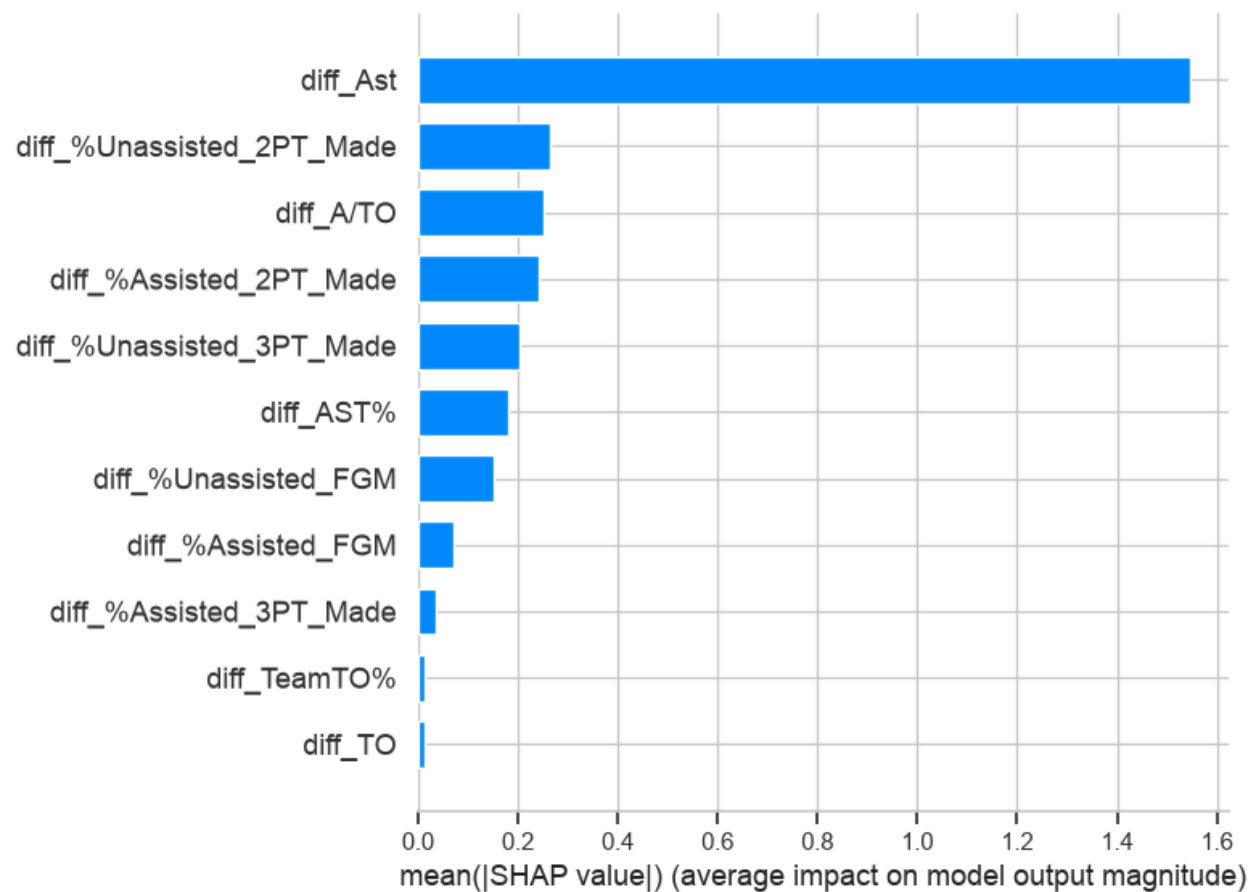


Figure S22: SHAP, playmaking (differential)

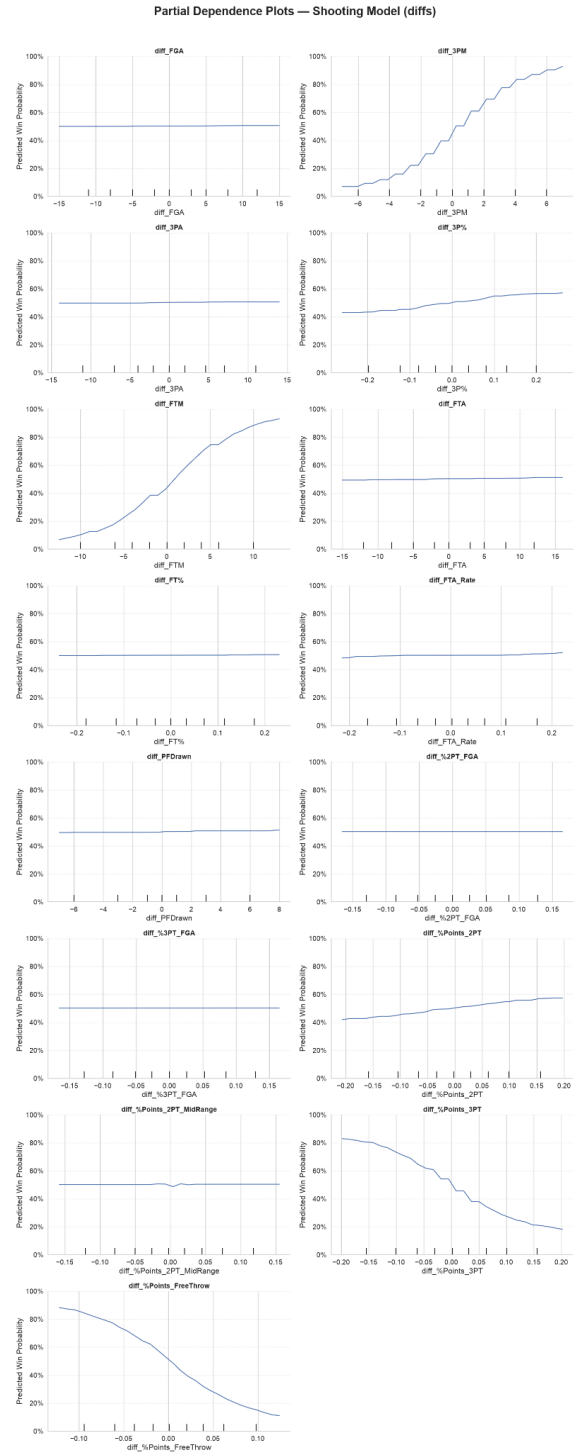


Figure S23: PDP, shooting (differential)

Partial Dependence Plots — Rebounding Model (difs)

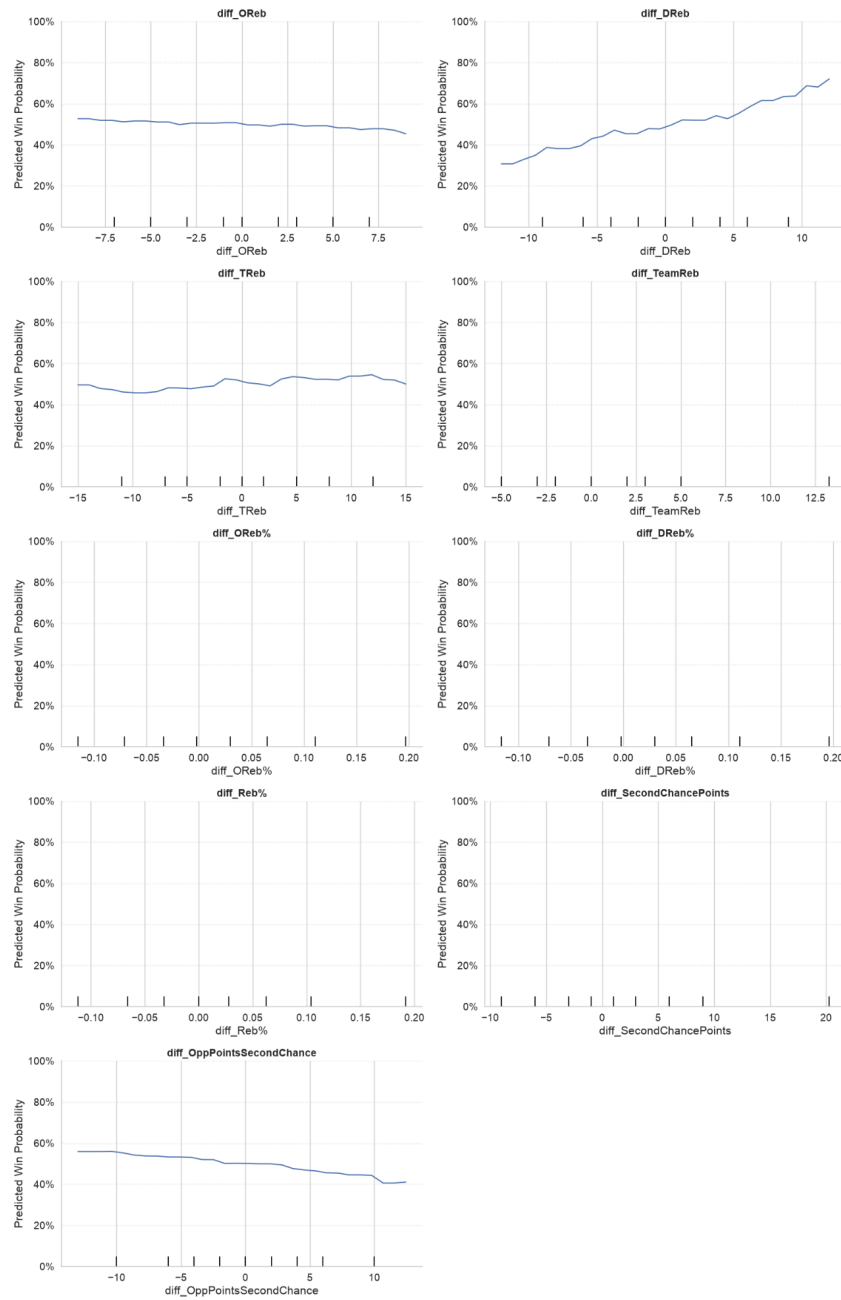


Figure S24: PDP, rebounding (differential)

Partial Dependence Plots — Scoring Model (difs)

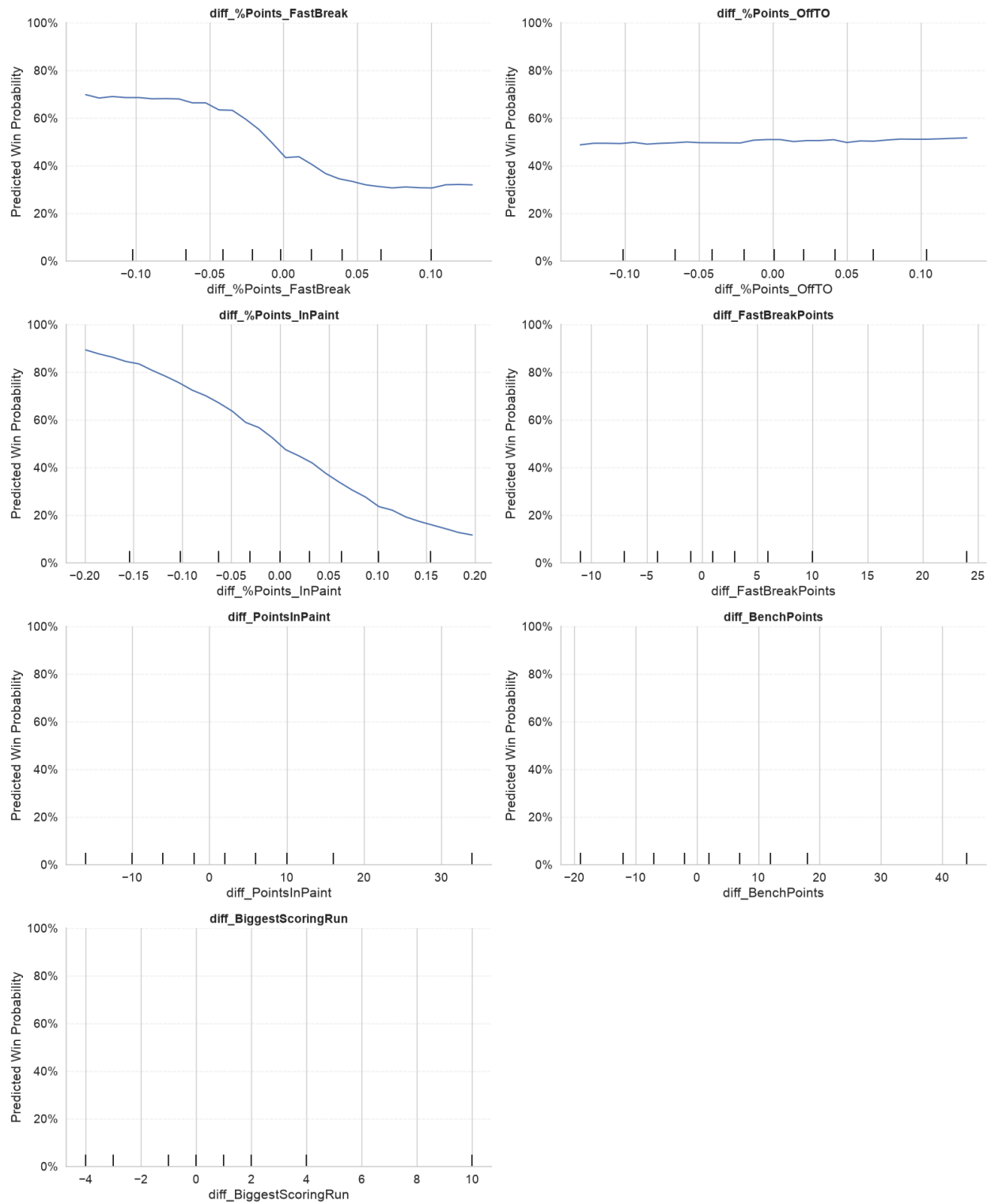


Figure S25: PDP, scoring (differential)

Partial Dependence Plots — Defense Model (diffs)

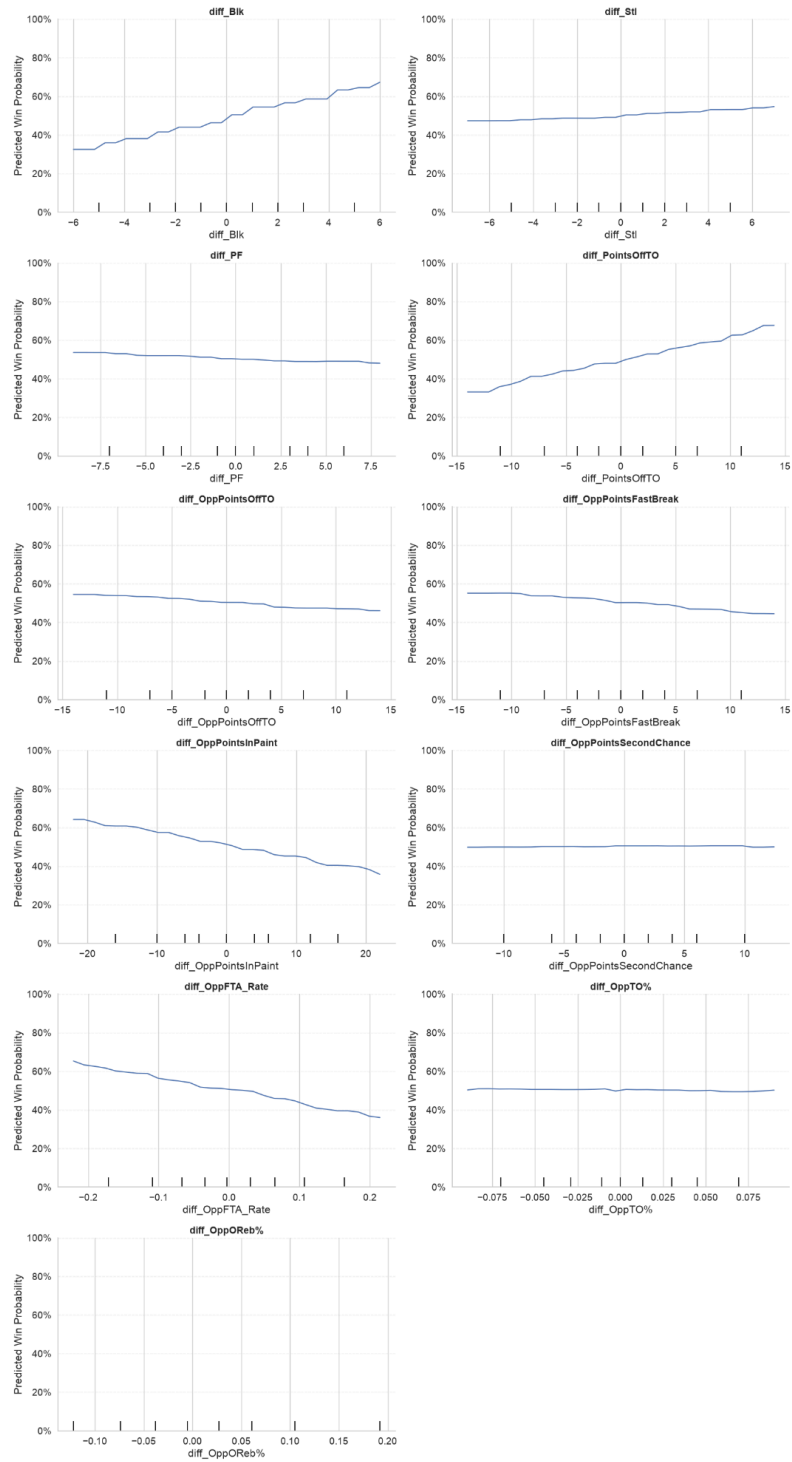


Figure S26: PDP, defense (differential)
Partial Dependence Plots — Pace Model (diffs)

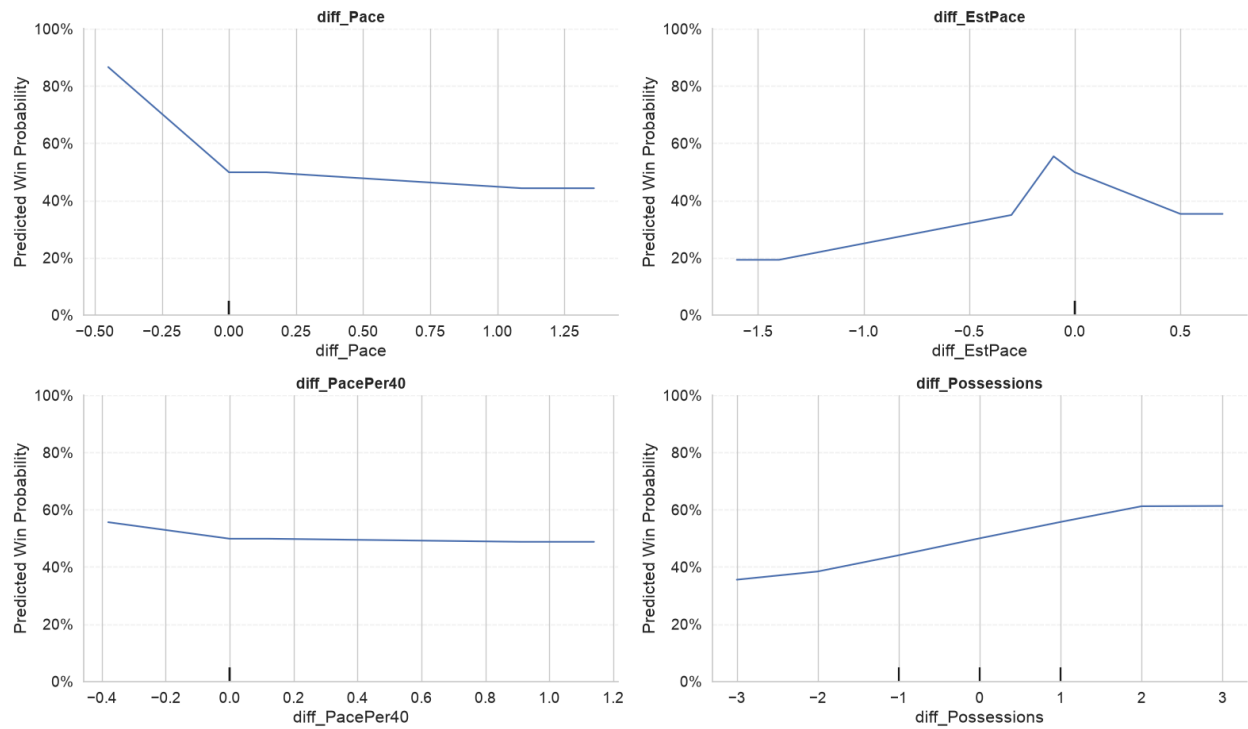


Figure S27: PDP, pace (differential)

2.5 Final Seven-Feature Model

The final model combines the seven most impactful, fully populated differentials. On the held-out test set (7,483 team-game rows) it reached 87.4% accuracy, AUC 0.9516, log loss 0.4293, and Brier score 0.1288. Mean absolute SHAP values (Figure S28) rank the features as follows.

Feature	Mean absolute SHAP
diff_DReb	0.386
diff_Ast	0.281
diff_PointsOffTO	0.254
diff_3PM	0.224
diff_FTM	0.196
diff_OppPointsInPaint	0.144
diff_Blkc	0.093

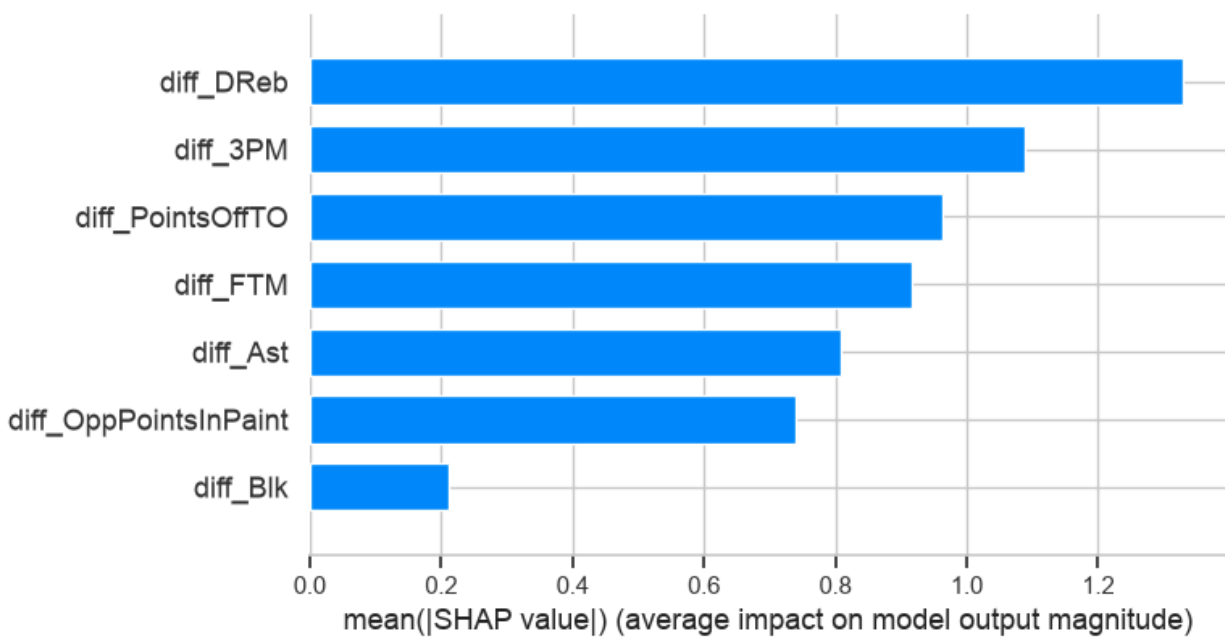


Figure S28: SHAP summary, final model

Partial Dependence Plots — Altogether Model

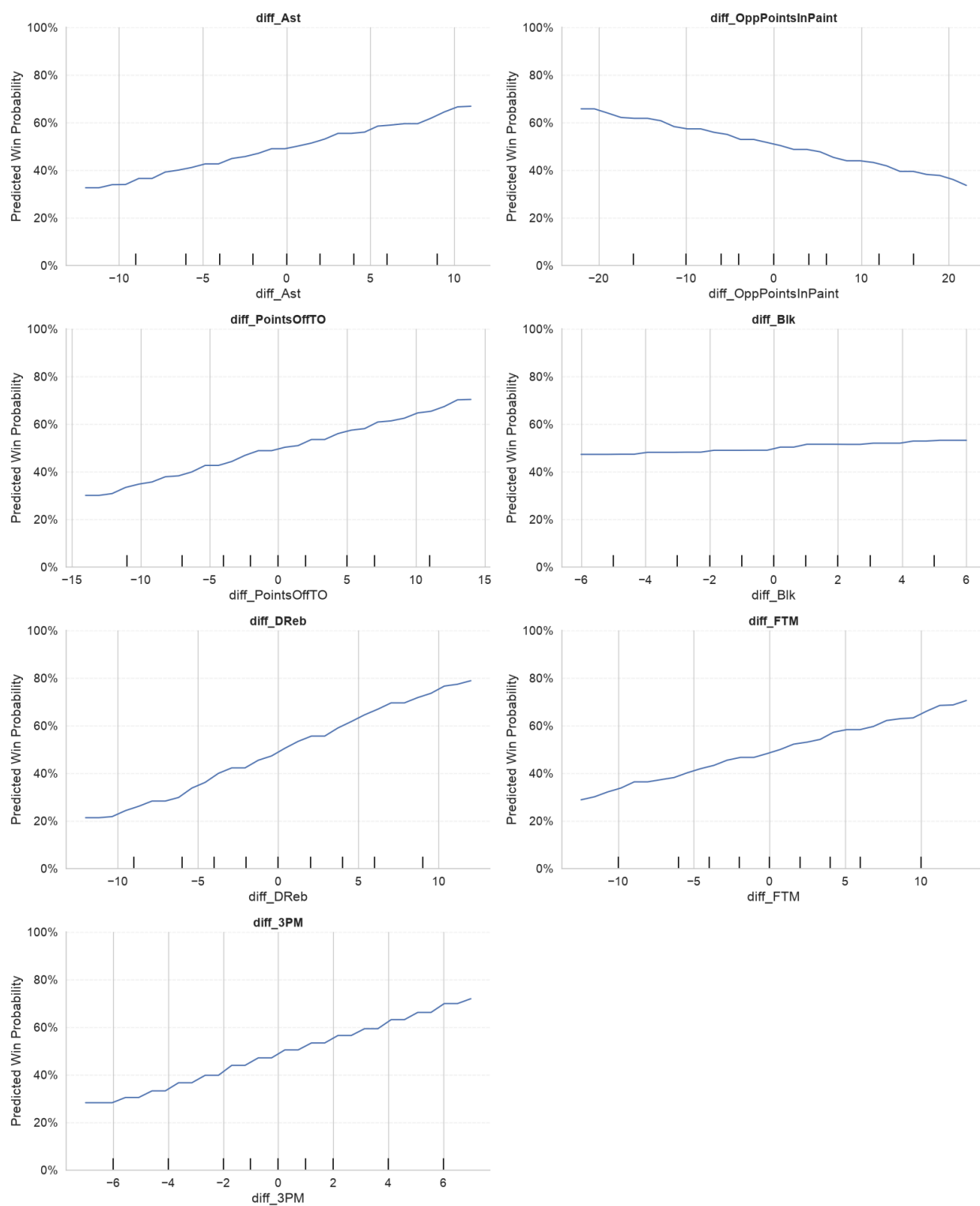


Figure S29: PDPs for each feature, final model

The training and validation log-loss curves (Figure S30) show whether the model is overfitting and where early stopping ended training. The ROC curve (Figure S31) and calibration curve (Figure S32) show discrimination and probability calibration. The confusion matrix (Figure S33) has 3,248 true negatives, 477 false positives, 464 false negatives, and 3,294 true positives, so errors are nearly balanced between the two classes.

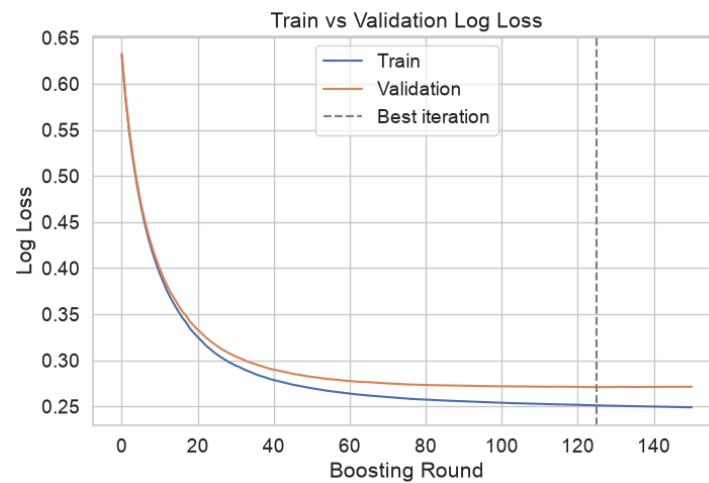


Figure S30: Training versus validation log loss by boosting round

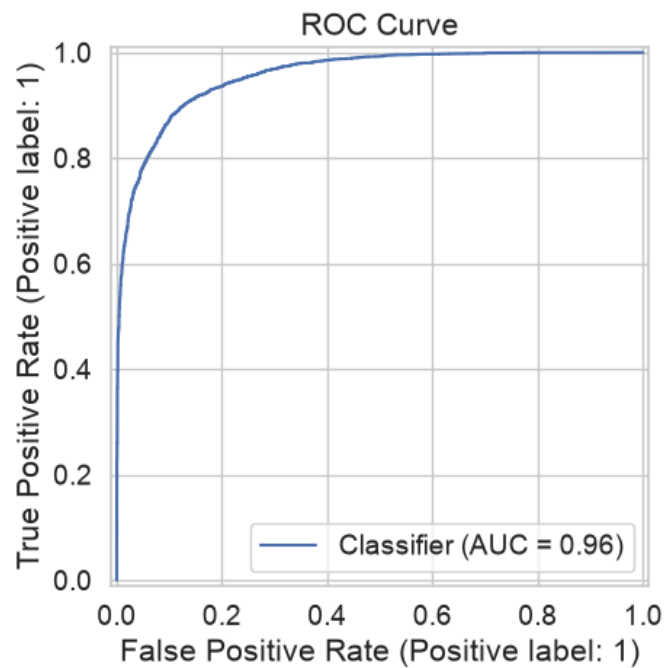


Figure S31: ROC curve, test set

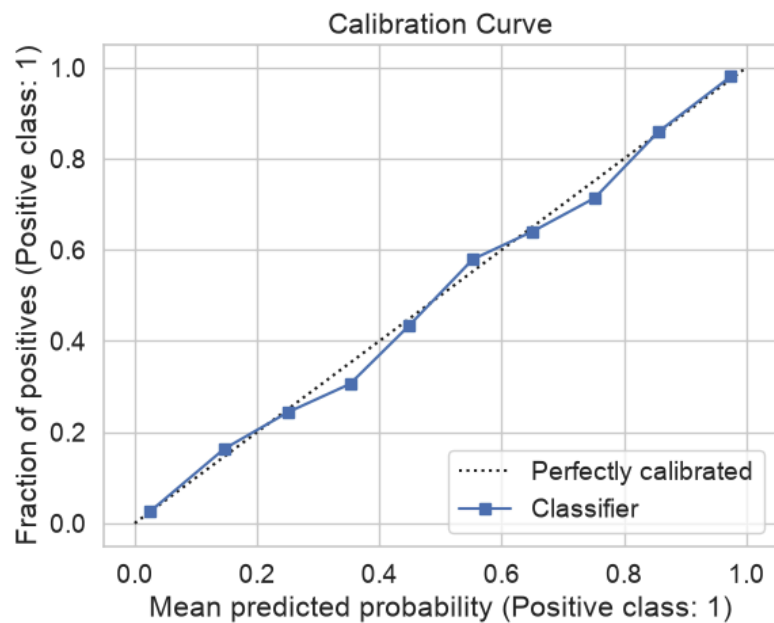


Figure S32: Calibration curve, test set

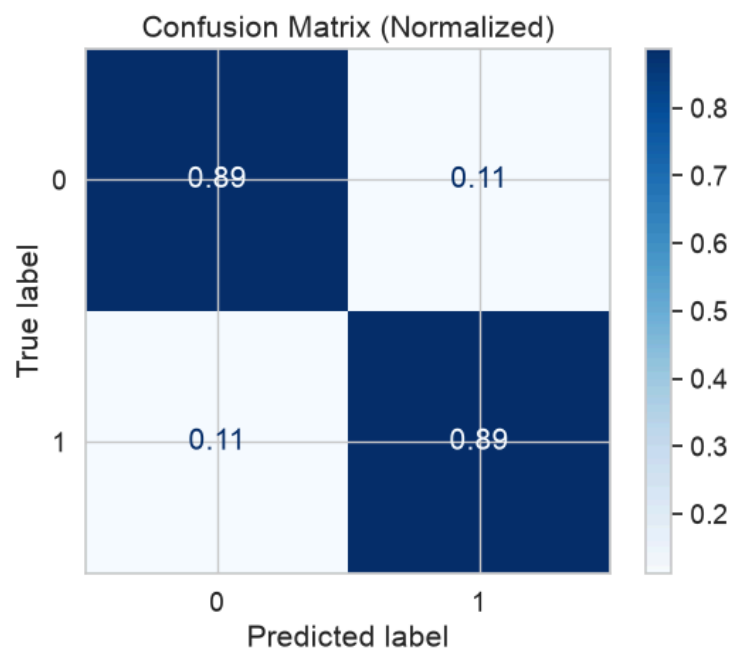


Figure S33: Confusion matrix, test set

3. Final Findings

3.1 Season Simulation and Vegas Comparison (main-text Section 4.5)

The simulation processed 1,200 regular-season games for 2026-27 (Section 1.16). Tables S34 (West) and S35 (East) give the full projected standings beside the preseason Vegas over/under (Rogers, n.d.). Projected win totals are the simulated totals after the two-game adjustment toward 41 described in Section 1.17.

Western Conference

Team	Model W	L	Win %	Vegas O/U	Model - Vegas
SAS	63	19	.768	59.5	+3.5
OKC	59	23	.720	62.5	-3.5
POR	57	25	.695	42.5	+14.5
LAL	55	27	.671	46.5	+8.5
GSW	46	36	.561	40.5	+5.5
DEN	41	41	.500	49.5	-8.5
MIN	41	41	.500	48.5	-7.5
MEM	34	48	.415	28.5	+5.5
NOP	31	51	.378	27.5	+3.5
PHX	30	52	.366	38.5	-8.5
HOU	27	55	.329	47.5	-20.5
SAC	26	56	.317	21.5	+4.5
UTA	25	57	.305	35.5	-10.5
DAL	16	66	.195	34.5	-18.5
LAC	11	71	.134	30.5	-19.5

Figure 34: Projected Western Conference Standings and Vegas Odds for 2027 Season

Eastern Conference

Team	Model W	L	Win %	Vegas O/U	Model - Vegas
NYK	61	21	.744	52.5	+8.5
MIA	57	25	.695	46.5	+10.5
PHI	57	25	.695	50.5	+6.5
BOS	55	27	.671	51.5	+3.5
CLE	49	33	.598	47.5	+1.5
ORL	46	36	.561	43.5	+2.5
IND	45	37	.549	44.5	+0.5
ATL	42	40	.512	43.5	-1.5
WAS	42	40	.512	34.5	+7.5
CHA	41	41	.500	37.5	+3.5
DET	38	44	.463	49.5	-11.5
TOR	34	48	.415	45.5	-11.5
MIL	27	55	.329	25.5	+1.5
CHI	26	56	.317	27.5	-1.5
BKN	18	64	.220	24.5	-6.5

Figure 35: Projected Eastern Conference Standings and Vegas Odds for 2027 Season

Projected standings. The model projects San Antonio (63 wins), Oklahoma City (59), Portland (57), and the Lakers (55) as the top Western teams, and New York (61), Miami (57), Philadelphia (57), and Boston (55) as the top Eastern teams. The lowest projections are the Clippers (11) and Mavericks (16) in the West and Brooklyn (18) and Chicago (26) in the East.

Agreement with Vegas. Model totals track the Vegas lines (Pearson $r = 0.77$) but are more spread out, ranging from 11 to 63 wins against 21.5 to 62.5 for Vegas. The mean absolute difference between the model and Vegas is 7.4 wins overall (West 9.5, East 5.2). Twelve of 30 teams are within 5 wins of their line and eight differ by more than 10. The model is above Vegas for 17 teams and below for 13, with a mean signed difference of -1.3 wins. That small negative offset is partly mechanical: the simulation covers 1,200 games, so model wins total 1,200, whereas a full 82-game season has 1,230 games (the Vegas lines sum to 1,238).

Largest deviations from Vegas (Figures 5 and 6). The largest positive deviations are Portland (+14.5; 57 projected wins against a 42.5 line), Miami (+10.5), New York (+8.5), the

Lakers (+8.5), Washington (+7.5), and Philadelphia (+6.5). The largest negative deviations are Houston (-20.5; 27 against 47.5), the Clippers (-19.5), Dallas (-18.5), Detroit (-11.5), Toronto (-11.5), and Utah (-10.5). Deviations are larger in the West than in the East. The full list is in Tables S2 and S3.

Biggest positive deviations

Team	Difference
POR	+14.5
MIA	+10.5
NYK	+8.5
LAL	+8.5
WAS	+7.5
PHI	+6.5
GSW	+5.5
MEM	+5.5

Figure 36: Biggest positive deviations from Vegas odds (model wins above the Vegas over/under)

Biggest negative deviations

Team	Difference
HOU	-20.5
LAC	-19.5
DAL	-18.5
DET	-11.5
TOR	-11.5
UTA	-10.5
DEN	-8.5
PHX	-8.5

Figure 37: Biggest negative deviations from Vegas odds (model wins below the Vegas over/under)

Comparison with prior-season results. The tables also list each team's win total from the preceding season [TODO: the "Actual Wins" column in the notebook matches 2025-26 results; confirm and relabel]. Because the 2026-27 season has not been played, this column is a reference point and not an accuracy test. Model totals differ from last season's results by 9.9 wins on average, against 5.6 for the Vegas lines (Table S4), and correlate 0.57 with last season's wins, against 0.82 for Vegas. The model therefore departs from last season's results more than the betting market does.

Limitations of this comparison.

- The simulation is deterministic and produces one win total per team. It gives no uncertainty interval.
- Simulated games total 1,200, not 1,230.
- Rotations fell back on placeholder players late in the season whenever rostered players had used up their projected games. Teams with thin projected availability are therefore likely to be projected low [TODO: log placeholder counts per team to check whether the largest negative deviations, such as Houston, the Clippers, and Dallas, are driven by this].
- Model quality can only be judged against 2026-27 results once the season is complete.

4. References

Anthropic. (2026). *Claude* (Sonnet 5.5) [Large language model]. <https://claude.ai> (n.d.). nba2kapi. Retrieved October 4, 2026, from <https://www.nba2kapi.com/playground?era=curr&sort=overall%3Adesc>

Chaparro, E., Zukiewicz, P., & Wyss, M. (n.d.). NBA: predicting win probability with deep learning. Medium. Retrieved October 4, 2026, from <https://medium.com/@zukiewicz.piotr/estimating-nba-in-game-win-probability-with-a-not-so-deep-neural-network-f6731a2e0ea9>

How MLB Win Projection Models Actually Work. (2025, April 9). Youtube. Retrieved October 4, 2026, from <https://www.youtube.com/watch?v=oKMm5pmw5bo>

Moore, E. A. (n.d.). NBA Dataset: Box Scores and Stats (1947 - Today). Kaggle. Retrieved October 4, 2026, from <https://www.kaggle.com/datasets/eoinamoore/historical-nba-data-and-player-box-scores/data>

NBA Win Probability: Methodology and Live Calculation. (n.d.). Hoops Junkie. Retrieved October 4, 2026, from <https://hoopsjunkie.io/methodology/win-probability>

Noh, S. (2026, July 29). NBA over/under predictions: Win totals and best unders for 2026-27 season, including Wizards, Pacers, and Mavs. Yahoo Sports. Retrieved October 4, 2026, from

https://sports.yahoo.com/articles/nba-over-under-predictions-win-121446599.html?guccounter=1&guce_referrer=aHR0cHM6Ly93d3cuZ29vZ2x1LmNvbS8&guce_referrer_sig=AQAAABFIAp0CC-BjHgml8I9mfoSdO9ikoIrfaVISSVEqV4HJq9NSHg976SKF82p1HHZKMAbuWhdTv98SJdUwAJ_fX6a5W27uy9Q

OpenAI. (2026). *ChatGPT* (GPT-5) [Large language model]. <https://chatgpt.com>

Palmieri, E. (n.d.). NBA - Player Stats - Season 24/25. Kaggle. Retrieved October 4, 2026, from <https://www.kaggle.com/datasets/eduardopalmieri/nba-player-stats-season-2425>

PECOTA Projections. (2026, March 26). Baseball Prospectus. Retrieved October 4, 2026, from <https://www.baseballprospectus.com/pecota-projections/>

Rogers, K. (n.d.). 2024-25 NBA Win Totals Odds - Find Win Totals Future Odds. Vegas Insider. Retrieved October 4, 2026, from <https://www.vegasinsider.com/nba/odds/win-totals/>

swar/nba_api: An API Client package to access the APIs for NBA.com · GitHub. (n.d.). GitHub. Retrieved October 4, 2026, from https://github.com/swar/nba_api

XGBoost. (2026, March 19). GeeksforGeeks. Retrieved October 4, 2026, from <https://www.geeksforgeeks.org/machine-learning/xgboost/>